

ฐานข้อมูลขั้นสูง · GRAPH DATABASE

Club Recommendation Graph ด้วย Neo4j

ระบบแนะนำชมรมให้นักเรียน จากเครือข่ายเพื่อนและความสนใจ
ส่วนตัว โดยใช้ Cypher บนฐานข้อมูลกราฟ

ชื่อ-สกุล

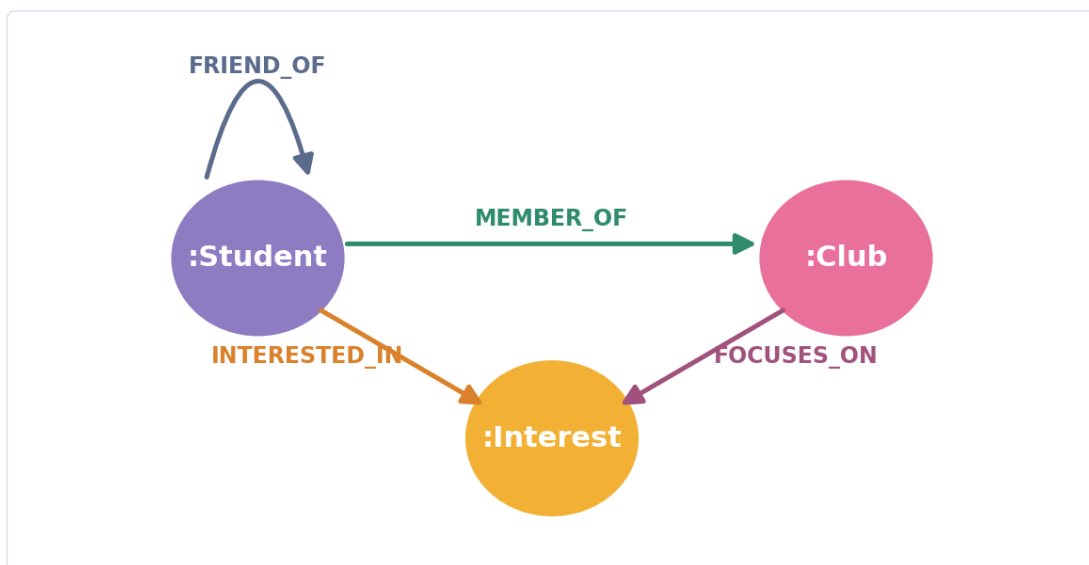
รหัสนักศึกษา

วันที่ส่ง 4 ตุลาคม 2569

1 ออกแบบโมเดลกราฟ

โจทย์คือแนะนำชมรมให้นักเรียนคนหนึ่ง โดยดูว่า เพื่อนของเขาอยู่ชมรมไหนกันบ้าง และ ชมรมนั้นตรงกับสิ่งที่เขาสนใจไหม จึงแยกข้อมูลเป็น 3 Node และ 4 Relationship ดังนี้

องค์ประกอบ	Property	เหตุผลที่ออกแบบแบบนี้
:Student	student_id, name, grade	ตัวผู้ใช้ที่เป็นทั้งผู้รับคำแนะนำและเป็นเพื่อนของคนอื่น
:Club	club_id, name	สิ่งที่ต้องการแนะนำ
:Interest	interest_id, name	หลายคนและหลายชมรมใช้ร่วมกันได้ จึงแยกเป็น Node แทนการเก็บเป็นข้อความใน Property
FRIEND_OF	Student → Student	เส้นทางหลักของคำแนะนำ (เก็บทิศเดียว แต่ query แบบไม่สนใจทิศ)
MEMBER_OF	Student → Club	ใครอยู่ชมรมไหน
INTERESTED_IN	Student → Interest	สิ่งที่นักเรียนสนใจ
FOCUSES_ON	Club → Interest	สิ่งที่ชมรมเน้น ใช้จับคู่กับความสนใจของนักเรียน



รูปที่ 1 โครงสร้าง (schema) ของกราฟชมรม

2 สร้าง Constraint และข้อมูลตัวอย่าง

ก่อนใส่ข้อมูล สร้าง Constraint ให้รหัสของแต่ละ Node ห้ามซ้ำ เพื่อป้องกันการสร้าง Node ซ้ำโดยไม่ตั้งใจ

```
CREATE CONSTRAINT student_id_unique IF NOT EXISTS
FOR (s:Student) REQUIRE s.student_id IS UNIQUE;

CREATE CONSTRAINT club_id_unique IF NOT EXISTS
FOR (c:Club) REQUIRE c.club_id IS UNIQUE;

CREATE CONSTRAINT interest_id_unique IF NOT EXISTS
FOR (i:Interest) REQUIRE i.interest_id IS UNIQUE;
```

จากนั้นสร้างข้อมูลตัวอย่าง นักเรียน 7 คน ชมรม 5 ชมรม ความสนใจ 4 อย่าง ตัวที่ใช้ทดสอบคือ **Fah (S001)** ซึ่งเป็นสมาชิก Drama Club อยู่แล้ว สนใจ Technology กับ Strategy และมีเพื่อน 4 คน

```
CREATE
// นักเรียน 7 คน
(s1:Student {student_id:'S001', name:'Fah', grade:'M.6'}),
(s2:Student {student_id:'S002', name:'Poom', grade:'M.6'}),
(s3:Student {student_id:'S003', name:'Praew', grade:'M.5'}),
(s4:Student {student_id:'S004', name:'Thee', grade:'M.5'}),
(s5:Student {student_id:'S005', name:'Mint', grade:'M.6'}),
(s6:Student {student_id:'S006', name:'Noon', grade:'M.5'}),
(s7:Student {student_id:'S007', name:'Kla', grade:'M.4'}),
// ชมรม 5 ชมรม
(c1:Club {club_id:'C01', name:'Robotics Club'}),
(c2:Club {club_id:'C02', name:'Chess Club'}),
(c3:Club {club_id:'C03', name:'Drama Club'}),
(c4:Club {club_id:'C04', name:'Photography Club'}),
(c5:Club {club_id:'C05', name:'Football Club'}),
// ความสนใจ 4 อย่าง
(i1:Interest {interest_id:'I01', name:'Technology'}),
(i2:Interest {interest_id:'I02', name:'Strategy'}),
(i3:Interest {interest_id:'I03', name:'Art'}),
(i4:Interest {interest_id:'I04', name:'Sport'}),
// เพื่อน (เก็บรหัสเดียว ตอน query ค่อยมองไม่สนรหัส)
(s1)-[:FRIEND_OF]->(s2), (s1)-[:FRIEND_OF]->(s3),
(s1)-[:FRIEND_OF]->(s4), (s1)-[:FRIEND_OF]->(s5),
(s2)-[:FRIEND_OF]->(s3), (s3)-[:FRIEND_OF]->(s5),
(s4)-[:FRIEND_OF]->(s6), (s6)-[:FRIEND_OF]->(s7),
// สมาชิกชมรม
(s1)-[:MEMBER_OF]->(c3),
(s2)-[:MEMBER_OF]->(c1), (s2)-[:MEMBER_OF]->(c2),
(s3)-[:MEMBER_OF]->(c3), (s3)-[:MEMBER_OF]->(c4),
(s4)-[:MEMBER_OF]->(c1), (s4)-[:MEMBER_OF]->(c5),
(s5)-[:MEMBER_OF]->(c4),
(s6)-[:MEMBER_OF]->(c5),
(s7)-[:MEMBER_OF]->(c1),
// ความสนใจของนักเรียน
(s1)-[:INTERESTED_IN]->(i1), (s1)-[:INTERESTED_IN]->(i2),
(s2)-[:INTERESTED_IN]->(i1),
(s3)-[:INTERESTED_IN]->(i3),
(s4)-[:INTERESTED_IN]->(i2), (s4)-[:INTERESTED_IN]->(i4),
(s5)-[:INTERESTED_IN]->(i3),
(s6)-[:INTERESTED_IN]->(i4),
(s7)-[:INTERESTED_IN]->(i1),
// จุดเน้นของชมรม
(c1)-[:FOCUSES_ON]->(i1), (c1)-[:FOCUSES_ON]->(i2),
(c2)-[:FOCUSES_ON]->(i2),
(c3)-[:FOCUSES_ON]->(i3),
(c4)-[:FOCUSES_ON]->(i3),
(c5)-[:FOCUSES_ON]->(i4);
```

3

ตรวจสอบจำนวนข้อมูล

นับ Node และ Relationship ที่สร้างแล้วเทียบกับที่ออกแบบไว้ได้ Node รวม 16 และ Relationship รวม 33

```
MATCH (n)
RETURN labels(n)[0] AS label, count(*) AS total
ORDER BY label;
```

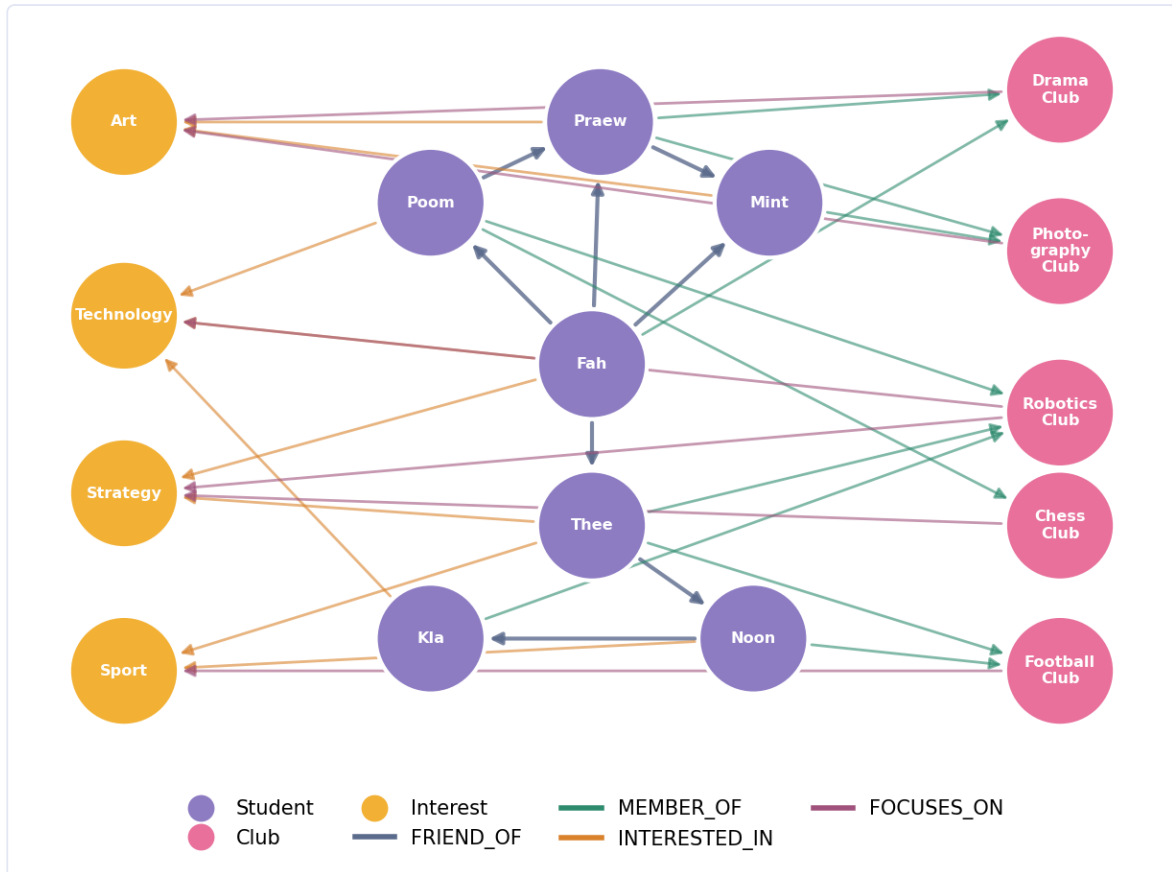
ผลลัพธ์ (3 แถว)

label	total
Club	5
Interest	4
Student	7

```
MATCH ()-[r]->()
RETURN type(r) AS relationship, count(*) AS total
ORDER BY relationship;
```

ผลลัพธ์ (4 แถว)

relationship	total
FOCUSES_ON	6
FRIEND_OF	8
INTERESTED_IN	9
MEMBER_OF	10



รูปที่ 2 กราฟทั้งหมดหลังสร้างข้อมูล (Interest ซ้าย, Student กลาง, Club ขวา)

4 ค้นหาเพื่อนของ Fah อยู่ชมรมไหนบ้าง

เดิน 2 ทอดจาก Fah ไปหาเพื่อน แล้วไปต่อที่ชมรมของเพื่อน ใช้เครื่องหมาย - ไม่ใส่ทิศ เพราะเราเก็บ FRIEND_OF ไว้ทิศเดียว

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
-[:MEMBER_OF]->(club:Club)
RETURN friend.name AS friend_name,
club.name AS club_name
ORDER BY friend_name, club_name;
```

ผลลัพธ์ (7 แถว)

friend_name	club_name
Mint	Photography Club
Poom	Chess Club
Poom	Robotics Club
Praew	Drama Club
Praew	Photography Club
Thee	Football Club
Thee	Robotics Club

5

นับจำนวนเพื่อนในแต่ละชมรม

เปลี่ยนจากรายคนเป็นรายชมรม ใช้ `count(DISTINCT friend)` เป็นคะแนนเพื่อน และ `collect` ไว้ดูว่าเป็นเพื่อนคนไหนบ้าง

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
-[:MEMBER_OF]->(club:Club)
RETURN club.club_id AS club_id,
club.name AS club_name,
count(DISTINCT friend) AS friend_score,
collect(DISTINCT friend.name) AS friends
ORDER BY friend_score DESC, club_name;
```

ผลลัพธ์ (5 แถว)

club_id	club_name	friend_score	friends
C04	Photography Club	2	Mint, Praew
C01	Robotics Club	2	Poom, Thee
C02	Chess Club	1	Poom
C03	Drama Club	1	Praew
C05	Football Club	1	Thee

สังเกตว่า Photography Club กับ Robotics Club ได้ 2 เท่ากัน และ Drama Club ก็ติดมาด้วยทั้งที่ Fah เป็นสมาชิกอยู่แล้ว ซึ่งต้องกรองออกในขั้นถัดไป

6

ตัดชมรมที่ Fah เป็นสมาชิกอยู่แล้ว

ใช้ `WHERE NOT EXISTS` ตรวจสอบว่าระหว่าง Fah กับชมรมนั้นยังไม่มีเส้น `MEMBER_OF` ถ้ามีแล้วก็ไม่ต้องแนะนำซ้ำ

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
-[:MEMBER_OF]->(club:Club)
WHERE NOT EXISTS { (me)-[:MEMBER_OF]->(club) }
RETURN club.name AS club_name,
count(DISTINCT friend) AS friend_score
ORDER BY friend_score DESC, club_name;
```

ผลลัพธ์ (4 แถว)

club_name	friend_score
Photography Club	2
Robotics Club	2
Chess Club	1
Football Club	1

7

เพิ่มคะแนนจากความสนใจ

ถ้าชมรมเน้นเรื่องที่ Fah สนใจ ให้บวกเพิ่มความสนใจละ 1 คะแนน โดยรวมคะแนนเพื่อนเป็นรายชมรมก่อนด้วย WITH แล้วค่อย OPTIONAL MATCH หาความสนใจที่ตรงกัน ถ้าไม่ได้รวมก่อน แล้วยจะคูณกันและนับเพี้ยนได้ ส่วน OPTIONAL ช่วยให้ชมรมที่ไม่ตรงความสนใจยังอยู่ในผลลัพธ์ (โบนัส = 0)

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
-[:MEMBER_OF]->(club:Club)
WHERE NOT EXISTS { (me)-[:MEMBER_OF]->(club) }
WITH me, club, count(DISTINCT friend) AS friend_score // รวมเป็นรายชมรมก่อน

OPTIONAL MATCH (me)-[:INTERESTED_IN]->(i:Interest)-[:FOCUSES_ON]-(club)
WITH club, friend_score, count(i) AS interest_bonus // 1 ความสนใจที่ตรง = +1

RETURN club.name AS club_name,
       friend_score,
       interest_bonus,
       friend_score + interest_bonus AS total_score
ORDER BY total_score DESC, club_name;
```

ผลลัพธ์ (4 แถว)

club_name	friend_score	interest_bonus	total_score
Robotics Club	2	2	4
Chess Club	1	1	2
Photography Club	2	0	2
Football Club	1	0	1

ผลที่ได้: ถ้าดูแค่เพื่อน Photography Club จะได้อันดับ 1 (2 คะแนนเท่ากับ Robotics แต่ชื่อขึ้นก่อนตามตัวอักษร) แต่พอรวมความสนใจ Robotics Club ได้โบนัสถึง 2 (Technology และ Strategy) คะแนนรวมเป็น 4 จึงขึ้นอันดับ 1 อย่างชัดเจน

8

แสดงเหตุผลของคำแนะนำ

คะแนนอย่างเดียวย่ออ่านแล้วไม่รู้ที่มา เลยเก็บชื่อเพื่อนและความสนใจที่ตรงกันไว้เป็นลิสต์ ผู้ใช้จะเห็นว่า "ทำไมถึงแนะนำชมรมนี้"

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
-[:MEMBER_OF]->(club:Club)
WHERE NOT EXISTS { (me)-[:MEMBER_OF]->(club) }
WITH me, club, collect(DISTINCT friend.name) AS friends

OPTIONAL MATCH (me)-[:INTERESTED_IN]->(i:Interest)-[:FOCUSES_ON]-(club)
WITH club, friends, collect(i.name) AS matched_interests

RETURN club.name AS club_name,
       friends,
       matched_interests,
       size(friends) + size(matched_interests) AS total_score
ORDER BY total_score DESC, club_name;
```

ผลลัพธ์ (4 แถว)

club_name	friends	matched_interests	total_score
Robotics Club	[Poom, Thee]	[Strategy, Technology]	4
Chess Club	[Poom]	[Strategy]	2
Photography Club	[Mint, Praew]	[]	2
Football Club	[Thee]	[]	1

9 เขียนผลลัพธ์กลับเข้ากราฟ (RECOMMENDED)

เก็บคำแนะนำเป็น Relationship ใหม่ RECOMMENDED พร้อมคะแนน จะได้เรียกใช้ซ้ำได้โดยไม่ต้องคำนวณใหม่ ใช้ MERGE เพื่อให้รันซ้ำแล้วไม่เกิดเส้นซ้ำ

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
      -[:MEMBER_OF]->(club:Club)
WHERE NOT EXISTS { (me)-[:MEMBER_OF]->(club) }
WITH me, club, count(DISTINCT friend) AS friend_score

OPTIONAL MATCH (me)-[:INTERESTED_IN]->(i:Interest)-[:FOCUSES_ON]-(club)
WITH me, club, friend_score, count(i) AS interest_bonus

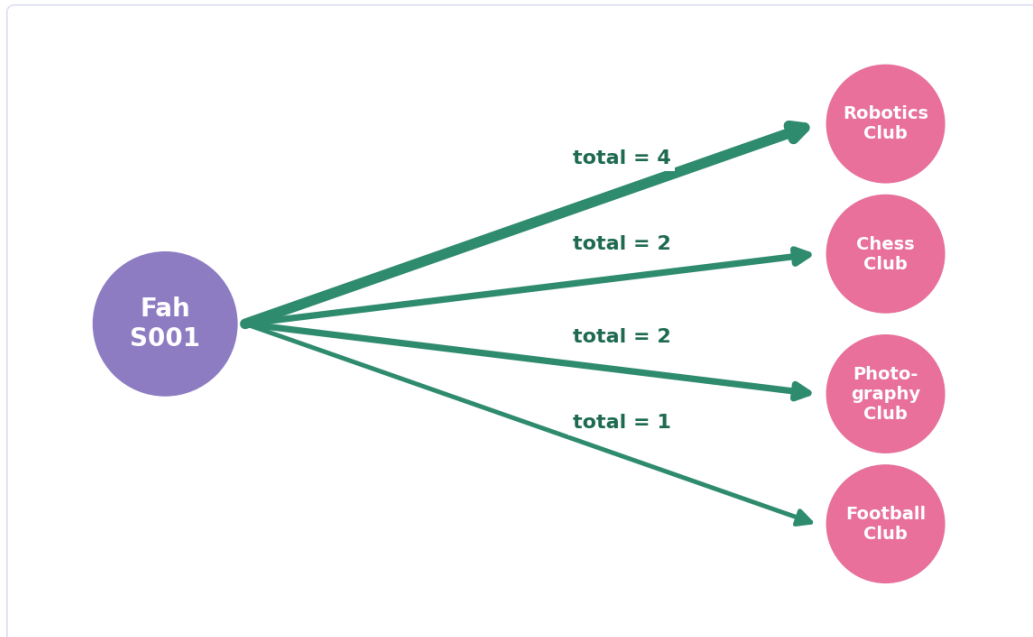
MERGE (me)-[r:RECOMMENDED]->(club)
SET r.friend_score = friend_score,
    r.interest_bonus = interest_bonus,
    r.total = friend_score + interest_bonus
RETURN me.name AS student, club.name AS club, r.total AS total
ORDER BY total DESC;
```

ผลลัพธ์ (4 แถว)

student	club	total
Fah	Robotics Club	4
Fah	Chess Club	2
Fah	Photography Club	2
Fah	Football Club	1

ดูผลเป็นกราฟด้วยคำสั่งนี้

```
MATCH p = (:Student {student_id:'S001'})-[:RECOMMENDED]->(:Club)
RETURN p;
```



รูปที่ 3 เส้น RECOMMENDED จาก Fah ไปยังชมรมที่แนะนำ (เส้นหนา = คะแนนสูง)

10 ทำเป็นฟังก์ชัน Python เรียกใช้กับนักเรียนคนอื่น

เอา query ข้างบนมาห่อเป็นฟังก์ชัน รับ `student_id` แล้วคืนตาราง pandas ตามที่ฝึกใน Colab (ใช้ `driver` และ `DATABASE` ที่เชื่อมต่อไว้แล้ว)

```
def recommend_clubs(student_id):
    query = """
    MATCH (me:Student {student_id:$sid})-[:FRIEND_OF]-(friend:Student)
      -[:MEMBER_OF]->(club:Club)
    WHERE NOT EXISTS { (me)-[:MEMBER_OF]->(club) }
    WITH me, club, count(DISTINCT friend) AS friend_score

    OPTIONAL MATCH (me)-[:INTERESTED_IN]->(i:Interest)<-[:FOCUSES_ON]-(club)
    WITH club, friend_score, count(i) AS interest_bonus

    RETURN club.name AS club,
           friend_score,
           interest_bonus,
           friend_score + interest_bonus AS total_score
    ORDER BY total_score DESC, club
    """
    result = driver.execute_query(query, sid=student_id, database_=DATABASE)
    return pd.DataFrame([r.data() for r in result.records])

recommend_clubs("S005")
recommend_clubs("S007")
```

recommend_clubs("S005") · Mint

ผลลัพธ์ (1 แถว)

club	friend_score	interest_bonus	total_score
Drama Club	2	1	3

recommend_clubs("S007") · Kla

ผลลัพธ์ (1 แถว)

club	friend_score	interest_bonus	total_score
Football Club	1	0	1

Mint มีเพื่อนคือ Fah กับ Praew ซึ่งอยู่ Drama Club ทั้งคู่ และ Mint สนใจ Art ที่ Drama เน้นพอดี คะแนนจึงเป็น 3 ส่วน Photography Club ถูกตัดออกเพราะ Mint เป็นสมาชิกอยู่แล้ว ส่วน Kla มีเพื่อนคนเดียวคือ Noon จึงได้คำแนะนำแค่ Football Club คะแนน 1 เพราะความสนใจของ Kla (Technology) ไม่ตรงกับชมรมนี้

11 สรุป

เส้นทางที่ใช้คิดคะแนนมี 2 ทาง คือ **เพื่อน** → **ชมรม** (Student-FRIEND_OF-Student-MEMBER_OF-Club) และ **ความสนใจ** → **ชมรม** (Student-INTERESTED_IN-Interest-FOCUSES_ON-Club) เมื่อรวมกันแล้วได้ลำดับที่เหมาะสมกับแต่ละคนมากกว่าการนับเพื่อนอย่างเดียว ดังที่เห็นในกรณีของ Fah ที่อันดับเปลี่ยนจาก Photography Club เป็น Robotics Club

ข้อดีของการเก็บเป็นกราฟคือเพิ่มเงื่อนไขใหม่ได้โดยแค่เพิ่ม pattern เข้าไปใน query เช่น ถ้าอยากให้เพื่อนที่อยู่ห้องเดียวกันมีน้ำหนักมากขึ้น ก็เพิ่มความสัมพันธ์หรือ Property แล้วเขียนเงื่อนไขต่อได้เลย โดยไม่ต้องแก้โครงสร้างตารางเหมือนฐานข้อมูลเชิงสัมพันธ์